

QUESTION 4

```
import random

# =====
# Genetic Algorithm for  $f(x) = -x^2 + 14x + 5$ 
#  $x$  in  $\{0, \dots, 15\}$  using 4-bit binary encoding
# =====

# -----
# Core Functions (Part a)
# -----

def decode(chromosome):
    return int("".join(map(str, chromosome)), 2)

def fitness(chromosome):
    x = decode(chromosome)
    return -x**2 + 14*x + 5

def roulette_select(population):
    total_fitness = sum(fitness(ind) for ind in population)

    if total_fitness == 0:
        return random.choice(population)

    r = random.random() * total_fitness

    cumulative = 0
    for ind in population:
        cumulative += fitness(ind)
        if cumulative >= r:
            return ind

    return population[-1]

def single_point_crossover(parent1, parent2, point):
    child1 = parent1[:point] + parent2[point:]
    child2 = parent2[:point] + parent1[point:]
    return child1, child2

def mutate(chromosome, pm):
    return [1 - bit if random.random() < pm else bit for bit in chromosome]
```

```

# -----
# GA Implementation (Part b)
# -----

def random_chromosome():
    return [random.randint(0, 1) for _ in range(4)]

def run_ga(pop_size, num_generations, pm, elitism, verbose=True):
    population = [random_chromosome() for _ in range(pop_size)]
    history = []

    for gen in range(num_generations):

        decoded = [decode(ind) for ind in population]
        fitness_vals = [fitness(ind) for ind in population]

        best_idx = fitness_vals.index(max(fitness_vals))
        best_ind = population[best_idx]
        best_fit = fitness_vals[best_idx]
        best_x = decoded[best_idx]

        history.append((gen, best_fit, best_x))

        if verbose:
            print(f"\nGeneration {gen}")
            for i in range(pop_size):
                print(f"{population[i]} -> x={decoded[i]}, f={fitness_vals[i]}")
            print(f"Best: {best_ind}, x={best_x}, f={best_fit}")

        new_population = []

        # Elitism: keep best individual unchanged
        if elitism:
            new_population.append(best_ind.copy())

        while len(new_population) < pop_size:
            p1 = roulette_select(population)
            p2 = roulette_select(population)

            point = random.randint(1, 3)

            c1, c2 = single_point_crossover(p1, p2, point)

```

```

        c1 = mutate(c1, pm)
        c2 = mutate(c2, pm)

        new_population.append(c1)
        if len(new_population) < pop_size:
            new_population.append(c2)

    population = new_population

    return history

# -----
# Testing Core Functions (Part a)
# -----

def test_core_functions():
    print("\n--- Testing Core Functions ---")

    test_chromosomes = [
        [0,1,1,0],
        [1,0,0,1],
        [1,1,0,0],
        [0,0,1,1]
    ]

    for c in test_chromosomes:
        x = decode(c)
        f = fitness(c)
        print(f"Chromosome: {c}, x = {x}, fitness = {f}")

# -----
# Experiments (Part c)
# -----

def experiment_elitism():
    print("\n--- Elitism vs No Elitism ---")

    for elitism in [False, True]:
        best_fitnesses = []
        found_optimum = 0
        gen_to_50 = []

```

```

for _ in range(30):
    results = run_ga(4, 20, 0.1, elitism, verbose=False)

    best_fit = max(r[1] for r in results)
    best_fitnesses.append(best_fit)

    if any(r[2] == 7 for r in results):
        found_optimum += 1

    for r in results:
        if r[1] >= 50:
            gen_to_50.append(r[0])
            break

avg_fit = sum(best_fitnesses) / 30
avg_gen = sum(gen_to_50)/len(gen_to_50) if gen_to_50 else None

print(f"\nElitism = {elitism}")
print(f"Average Best Fitness: {avg_fit}")
print(f"Runs reaching x=7: {found_optimum}/30")
print(f"Avg generation to reach f>=50: {avg_gen}")

def experiment_mutation_rates():
    print("\n--- Mutation Rate Experiment ---")

    for pm in [0.01, 0.1, 0.3, 0.5]:
        fits = []

        for _ in range(30):
            results = run_ga(4, 20, pm, False, verbose=False)
            fits.append(max(r[1] for r in results))

        avg_fit = sum(fits) / 30
        print(f"pm = {pm}, Average Best Fitness = {avg_fit}")

# -----
# Main Execution
# -----

if __name__ == "__main__":

    # Part (a)

```

```
test_core_functions()
```

```
# Part (b)
```

```
print("\n--- Running GA (Required Run) ---")
```

```
run_ga(pop_size=4, num_generations=10, pm=0.1, elitism=False, verbose=True)
```

```
# Part (c)
```

```
experiment_elitism()
```

```
experiment_mutation_rates()
```

```
--- Testing Core Functions ---  
Chromosome: [0, 1, 1, 0], x = 6, fitness = 53  
Chromosome: [1, 0, 0, 1], x = 9, fitness = 50  
Chromosome: [1, 1, 0, 0], x = 12, fitness = 29  
Chromosome: [0, 0, 1, 1], x = 3, fitness = 38
```

```
--- Running GA (Required Run) ---
```

```
Generation 0
```

```
[0, 1, 1, 1] -> x=7, f=54  
[1, 0, 0, 0] -> x=8, f=53  
[0, 1, 0, 0] -> x=4, f=45  
[0, 0, 0, 0] -> x=0, f=5  
Best: [0, 1, 1, 1], x=7, f=54
```

```
Generation 1
```

```
[1, 0, 0, 0] -> x=8, f=53  
[1, 0, 0, 1] -> x=9, f=50  
[0, 1, 0, 1] -> x=5, f=50  
[1, 0, 0, 0] -> x=8, f=53  
Best: [1, 0, 0, 0], x=8, f=53
```

```
Generation 2
```

```
[1, 0, 0, 1] -> x=9, f=50  
[0, 1, 0, 0] -> x=4, f=45  
[1, 0, 0, 1] -> x=9, f=50  
[1, 0, 1, 0] -> x=10, f=45  
Best: [1, 0, 0, 1], x=9, f=50
```

Generation 3

[1, 0, 1, 0] -> x=10, f=45

[0, 0, 0, 1] -> x=1, f=18

[1, 0, 0, 0] -> x=8, f=53

[1, 0, 0, 0] -> x=8, f=53

Best: [1, 0, 0, 0], x=8, f=53

Generation 4

[0, 0, 0, 1] -> x=1, f=18

[0, 0, 0, 0] -> x=0, f=5

[1, 1, 0, 0] -> x=12, f=29

[1, 0, 0, 0] -> x=8, f=53

Best: [1, 0, 0, 0], x=8, f=53

Generation 5

[1, 0, 0, 0] -> x=8, f=53

[1, 0, 0, 0] -> x=8, f=53

[1, 0, 0, 0] -> x=8, f=53

[1, 0, 0, 0] -> x=8, f=53

Best: [1, 0, 0, 0], x=8, f=53

Generation 6

[1, 1, 0, 0] -> x=12, f=29

[1, 0, 0, 0] -> x=8, f=53

[1, 0, 0, 0] -> x=8, f=53

[0, 0, 0, 0] -> x=0, f=5

Best: [1, 0, 0, 0], x=8, f=53

Generation 7

[1, 0, 0, 0] -> x=8, f=53

[1, 0, 0, 0] -> x=8, f=53

[1, 0, 0, 0] -> x=8, f=53

[0, 0, 0, 1] -> x=1, f=18

Best: [1, 0, 0, 0], x=8, f=53

Generation 8

[1, 0, 0, 1] -> x=9, f=50

[0, 0, 0, 0] -> x=0, f=5

[1, 1, 0, 1] -> x=13, f=18

[0, 0, 0, 0] -> x=0, f=5

Best: [1, 0, 0, 1], x=9, f=50

Generation 9

[1, 0, 0, 1] -> x=9, f=50

[1, 1, 0, 0] -> x=12, f=29

[1, 0, 1, 1] -> x=11, f=38

[1, 1, 0, 1] -> x=13, f=18

Best: [1, 0, 0, 1], x=9, f=50

```
--- Elitism vs No Elitism ---
```

```
Elitism = False
```

```
Average Best Fitness: 53.766666666666666
```

```
Runs reaching x=7: 23/30
```

```
Avg generation to reach f>=50: 0.5
```

```
Elitism = True
```

```
Average Best Fitness: 53.733333333333334
```

```
Runs reaching x=7: 22/30
```

```
Avg generation to reach f>=50: 0.4
```

```
--- Mutation Rate Experiment ---
```

```
pm = 0.01, Average Best Fitness = 52.46666666666667
```

```
pm = 0.1, Average Best Fitness = 53.733333333333334
```

```
pm = 0.3, Average Best Fitness = 53.96666666666667
```

```
pm = 0.5, Average Best Fitness = 54.0
```

- **Elitism effects:**

- Elitism slightly slows early convergence but improves stability of best solutions.
- Without elitism, the GA often reaches high fitness quickly because all individuals can change freely, but the best solution can sometimes be lost due to stochastic effects.
- With elitism, the best individual is preserved, so although fewer runs immediately reach the optimal $x=7$, the overall best fitness is maintained more consistently.

- **Mutation rate effects:**

- Low mutation ($p_m = 0.01$) is too conservative; the GA may get stuck in local optima (average best fitness ~52.57).
- Moderate mutation ($p_m = 0.1$) balances exploration and exploitation well, allowing the population to escape local optima and converge to the global maximum (average best fitness 53.73–54).
- Very high mutation ($p_m = 0.5$) introduces excessive randomness, essentially turning the GA into a near-random search. It can still occasionally reach the optimum, but the population fluctuates too much, reducing convergence stability.

- **Best mutation rate:**

- $p_m = 0.3$ or 0.5 produced the highest average best fitness (54.0), showing that moderate to high mutation helped escape local maxima in this small search space.
- However, in larger or more complex problems, very high mutation would be harmful because it prevents exploitation of good solutions.

(d) Manual Calculation

Step 1: Decode + Fitness

Individual Chromosome x f(x)

P1	[0,1,1,0]	6	53
P2	[1,0,0,1]	9	50
P3	[1,1,0,0]	12	29
P4	[0,0,1,1]	3	38

Total fitness = $53 + 50 + 29 + 38 = 170$

Step 2: Selection Probabilities

Individual Probability

P1	$53/170 = 0.312$
P2	$50/170 = 0.294$
P3	$29/170 = 0.171$
P4	$38/170 = 0.223$

Step 3: Cumulative Probabilities

Individual Range

P1	0 – 0.312
P2	0.312 – 0.606
P3	0.606 – 0.777
P4	0.777 – 1

Step 4: Selection (Using r values)

- $r1 = 0.12 \rightarrow \mathbf{P1}$
- $r2 = 0.47 \rightarrow \mathbf{P2}$
Pair 1: (P1, P2)
- $r3 = 0.68 \rightarrow \mathbf{P3}$
- $r4 = 0.91 \rightarrow \mathbf{P4}$
Pair 2: (P3, P4)

Step 5: Crossover (point = 2)

Pair 1:

P1 = 0110

P2 = 1001

$\rightarrow O1 = 01|01 = \mathbf{0101 (5)}$

$\rightarrow O2 = 10|10 = \mathbf{1010 (10)}$

Pair 2:

P3 = 1100

P4 = 0011

$\rightarrow O3 = 11|11 = \mathbf{1111 (15)}$

$\rightarrow O4 = 00|00 = \mathbf{0000 (0)}$

Step 6: Fitness

Offspring x f(x)

O1 5 50

O2 10 45

O3 15 20

O4 0 5

Step 7: Mutation (O1 only)

O1 = [0,1,0,1]

Random values = [0.08, 0.43, 0.91, 0.05], pm=0.1

- $0.08 < 0.1 \rightarrow \text{flip} \rightarrow 1$
- $0.43 > 0.1 \rightarrow \text{no change}$
- $0.91 > 0.1 \rightarrow \text{no change}$
- $0.05 < 0.1 \rightarrow \text{flip} \rightarrow 0$

Mutated O1 = **[1,1,0,0] = x=12**

f(12) = **29**